TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

UÇAN ARABA YARIŞMASI

FİNAL TASARIM RAPORU

İLERİ KATEGORİ

TAKIM ADI: RIDINY AVIATION

TAKIM ÜYELERİ: Nur YILDIRIM (Takım Kaptanı)

İzzet YILDIRIM, Ahmet Can KALAYCI, Buğra ŞAHİN

DANIŞMAN: Öğr. Gör. Mustafa ÖNCÜL

TAKIM ID: 318547

PROJE ADI: VULPES

İçindekiler

BÖLÜM-1 3
BÖLÜM-2 5
BÖLÜM-3 8
BÖLÜM-4 10
BÖLÜM-5 11
BÖLÜM-6 12
BÖLÜM-7 15
BÖLÜM-8 16
BÖLÜM-9 19
BÖLÜM-10 20
Kaynakça 21

BÖLÜM-1

Bu bölümde, hazırlanan senaryolarda kullanılan sensörler ve programlama dili ile ilgili bilgiler final tasarım şablonundaki isterlere göre sırasıyla açıklanmıştır.

I. Kod yazarken kullandığınız dil ve özelliklerini açıklayınız.

Kod yazımında Python programlama dili kullanılmıştır. Python, 1990'ların başında Guido Van Rossum tarafından herkesin rahatça kodlama yapabilmesi için tasarlanan bir high-level programlama dilidir[1]. Program, ilk ortaya çıktığından bu yana sürekli olarak yeni fonksiyonlar eklenerek güncellenmektedir. Açık kaynaklı bir dil olan Python'u diğer programlama dillerinden ayıran en önemli özellikleri kod parçacığının daha az satırda yazılmasına imkan tanıması, nispeten daha kolay olması, Mac, Windows ve Linux dahil olmak üzere mevcut tüm işletim sistemlerinde çalışabilmesidir. Aynı zamanda sırasıyla JavaScript ve C ++ 'dan yaklaşık üç ila beş kat daha hızlı kodlama yapılmasına imkan tanımaktadır[2]. Python programının başlıca kullanım alanlarına web geliştirme, yapay zeka ve makine öğrenimi, veri analizi, oyun geliştirme ve robotik uygulamaları örnek verilebilir.

II. Bu dil yerine başka hangi programlama dilini tercih ederdiniz? Nedeni ile açıklayınız.

Kod yazımında alternatif olarak MATLAB programlama dili belirlenmiştir. MATLAB programlama dilinde her bir veri elementi bir matris değerindedir ve kodların boyutunu düşürmek için vektörleştirilmiş işlemler kullanılmaktadır[3]. Böylece büyük boyuttaki kodların boyutları önemli ölçüde düşmektedir. Grafiksel bir programlama aracı olan Simulink sayesinde MATLAB'ın grafik sistemi etkileşim için optimize edilmiş çıkışlar sağlamaktadır. Bu sayede verinin grafiğe dönüşmesi kolaylaştırılarak rengi ile boyutu ayarlanabilmektedir. Simulink, çok alanlı simülasyon ve model tabanlı tasarım için bir blok diyagram ortamıdır[4]. Sistem düzeyinde tasarım, simülasyon, otomatik kod oluşturma ve gömülü sistemlerin sürekli test edilip doğrulanmasını desteklemektedir. MATLAB programının başlıca uygulama alanlarına sayısal lineer cebir hesaplamaları, veri analizi ve görselleştirmeleri, büyük verilerin grafiklerini oluşturma, farklı algoritmalar geliştirme, makine öğrenmesi, data bilimi, derin öğrenme, kullanıcılar için uygulama programlama arayüzü oluşturma ve tasarlanan ürünlerin simülasyonunu oluşturma örnek verilebilir.

III. Barometre sensörünün nasıl çalıştığını açıklayınız. GNSS ile barometre irtifa verilerinin farkı nedir?

Barometre temel olarak atmosferik basıncı yani hava aracının dışındaki basıncı ölçmeye yarayan alettir. Civalı barometreler, cam tüpteki cıvanın ağırlığını atmosfer basıncına karşı dengeleyerek çalışır. Civa seviyesi, rezervuarın üzerindeki havanın ağırlığına tam olarak eşit

olana kadar değişmeye devam eder. İkisi hareket etmeyi bırakıp dengelendiğinde basınç, dikey sütunda cıva yüksekliğindeki değer okunarak kaydedilir[5]. Nispeten küçük ebatlardaki hava araçlarında, genellikle kullanımda olan barometre türü aneroid barometredir. Aneroid barometrenin içinde küçük, esnek bir metal kutu bulunmaktadır. Bu kutunun içindeki hava dışarı pompalandığında dış hava basıncındaki küçük değişiklikler, metalinin genleşmesine veya büzülmesine neden olur. Genişleme veya büzülme hareketleri, içinde bir iğneyi hareket ettiren mekanik kolları çalıştırır. Bu hareketler, iğneyi barometre yüz kadranı çevresinde yukarı veya aşağı hareket ettirirken, basınç değişikliğinin görüntülenmesini sağlar[6]. Barometre, atmosfer basıncını ölçmenin yanısıra birçok farklı alanda da kullanılmaktadır. Bu alanlar genellikle açık hava basıncına bağlı olarak yapılan hesaplamalarla belirlenen verilerdir. Bunlara örnek olarak kısa süreli hava tahminleri ve basınca bağlı olarak yükseklik hesaplamaları verilebilir.

Küresel navigasyon uydu sistemleri anlamına gelen GNSS, hava aracının konumunu uzaydan geriden kestirme yöntemi ile bulunmasına yarayan alettir. Bu yöntemde uydu ile alıcı arası mesafe, sinyalin uydudan alıcıya ulaşana kadar geçen zamanın ışık hızı ile çarpılması sonucu bulunmaktadır. Daha sonra bu uydu-alıcı uzaklığı, uyduya eş zamanlı olarak yapılan gözlemlerle konum vektörüne dönüştürülmektedir. GNSS sistemleri GPS, GLONASS, GALILEO , BEIDOU/COMPASS, QZSS gibi birçok uydu bazlı sistemi içinde barındırmaktadır[7].

GNSS ile uçağın uydu sinyalleri kullanılarak irtifa ölçümü yapılabilmektedir. Bu ölçüm barometrik basınçtan tamamen bağımsızdır. Barometre ile irtifa ölçümü tamamen barometrik basınca dayalı olarak hesaplamalarla belirlenmektedir. Aynı zamanda GNSS irtifa verileri ortometrik irtifa yerine sadece elipsoidal irtifa vermektedir. Ortometrik irtifa, Dünya'nın yüzey mesafesindeki deniz seviyesindeki bir konumdan jeolojik özellikler dikkate alınarak hesaplanan dikey mesafedir. Elipsoidal irtifa ise Dünya'nın elips olduğu varsayılarak hesaplanan dikey mesafedir[8].

IV. İvmeölçer ve dönüölçer sensörlerinde genel hata karakteristikleri nelerdir? Bu karakteristiklerin özelliklerini açıklayınız.

İvmeölçer sensörü, ivmeyi ölçeye yarayan alettir. Sabit kayma hatası, orantı katsayısı hatası ve eksenel kaçıklık hatası ivmeölçerlerin başlıca deterministik hata karakteristikleridir. Sabit kayma kararsızlığı, orantı katsayı kararsızlığı ve rasgele sensör gürültüsü ivmeölçerin olasılıksal hata karakteristikleridir. Sabit kayma hatası, sensöre herhangi bir ivme uygulanmadığı durumda ölçülen değerdir. Sabit kayma kararsızlığı, ivmeölçerin çalışma sırasındaki değişiminden kaynaklı hataları içermektedir. Orantı katsayısı hatası, sensörün çıkışındaki değişim ile girişindeki değişim arasındaki orandır. Orantı katsayısı kararsızlığı, orantı katsayısındaki zamanla değişimi ifade etmektedir. Eksenel kaçıklık hatası, ivmeölçerlerin

birbirlerine dik monte edilememesi sonucunda oluşmaktadır. Rasgele sensör gürültüsünü yüksek frekanslı gürültü terimleri (nicemleme gürültüsü, hız rasgele yürüme ve Gauss-Markov gürültüsü) oluşturmaktadır[9].

Dönüölçer sensörü jiroskop, manyetometre ve ivmeölçer gibi 3 sensörün bir karışımı kullanılarak elde edilebilen vücut yönünü ve açısal oranını tam olarak hesaplamak ve raporlamak için kullanılan elektronik bir cihazdır. Sabit kayma hatası, orantı katsayısı hatası, eksenel kaçıklık hatası ve ivmeye bağlı sabit kayma hatası dönüölçerlerin başlıca deterministik hata karakteristikleridir. Sabit kayma kararsızlığı, orantı katsayı kararsızlığı ve rasgele sensör gürültüsü dönüölçerin olasılıksal hata karakteristikleridir. Sabit kayma hatası, dönüölçere herhangi bir giriş uygulanmadığı koşulda belirli bir süre ölçülen dönü değerlerinin ortalamasıyla belirlenmektedir. Sabit kayma kararsızlığı, çalışma sırasındaki değişiminden kaynaklı hataları içermektedir. Orantı katsayısı hatası, açısal hız girişi uygulanan eksen boyunca dönüölçer hassasiyetini oluşturmakta ve ppm (part per million) olarak ifade edilmektedir. Orantı katsayısı kararsızlığı, orantı katsayısındaki zamanla değişimi ifade etmektedir. Eksenel kaçıklık hatası, dönüölçerlerin yanlış hizalanmasından kaynaklanmaktadır. İvmeye bağlı sabit kayma hatası, MEMS dönüölçer yapısının doğrusal ivmeye karşı duyarlılık gösterip sensörün çıkışında bir kaymaya neden olmasıyla oluşmaktadır[9].

BÖLÜM-2

Bu bölümde, şartnamede yer alan Senaryo-1'de hava aracının B noktasına gidişi için tasarlanan algoritma için final tasarım şablonundaki sorular sırasıyla cevaplandırılmıştır.

I. Hava aracının gitmesi gereken yönü hesapladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

Algoritma	**Açıklama**
load("sehir.a.gridmap.mat","occGrid") show(occGrid)	Simülasyon ortamına aktarılacak şehir haritalaması görüntülenmiştir.

function GPS = ECEF2GPS(Pos)
x=Pos(1);
y=Pos(2);
z=Pos(3);
a = 6378137;
	Hava aracının başlangıç noktasının konumu GPS sensörüyle belirlenmiştir. Burada a ve e sabitleri WGS84 koordinat sisteminin elipsoidal sabitleridir.

e = 8.1819190842622e-2;
b = sqrt(a^2*(1-e^2));
ep = sqrt((a^2-b^2)/b^2);
p = sqrt(x^2+y^2);
th = atan2(a*z,b*p);
lon = atan2(y,x);
lat = atan((z+ep^2*b*(sin(th))^3)/(p-e^2*a*(cos(th))^3));
N = a/sqrt(1-e^2*(sin(lat))^2);
alt = p/cos(lat)-N;
k=abs(x)<1 & abs(y)<1;
alt(k) = abs(z(k))-b;
GPS=[lat,lon,alt];
Return
startNode=[GPS]
	b, ep, p, th, lon, lat, N, alt katsayıları konum için hesaplamalardır. k ve alt(k) katsayıları kesin kutupların yakınındaki sayısal kararsızlık için doğrulamadır[10].

sehir_a= csvread('sehir.a.veri.csv');
target=input('Nereye gitmek istiyorsunuz?','s');
endNode=sehir_a(target,'s');
	Hava aracının hedef noktası kullanıcı girdisiyle belirlenmiştir.

maxTreeSize=75000;	Cezeri'nin maksimum menzil tanımlanmıştır.
hold on plot(start(1),start(2),'ro') plot(goal(1),goal(2),'mo') r = 0.5; plot([start(1),start(1) + r*cos(start(3))],[start(2),start(2) + r*sin(start(3))],'r-') plot([goal(1),goal(1) + r*cos(goal(3))],[goal(2),goal(2) + r*sin(goal(3))],'m-') hold off	Hava aracının başlangıç ve bitiş konumu haritada işaretlenmiştir.

bounds = [occGrid.XWorldLimits; occGrid.YWorldLimits; [-pi pi]];
ss = stateSpaceDubins(bounds);
ss.MinTurningRadius = 3.74;
	stateSpaceDubins komutu kullanılarak ve durum sınırlarını belirterek aracın durum uzayını belirtilmiştir. Böylece durum sınırları içinde aracın

	yönlendirmesi Dubins eğrileriyle sınırlanmıştır. Cezeri için 3.74 metrelik dönüş yarıçapı tanımlanmıştır.

stateValidator = validatorOccupancyMap(ss);
stateValidator.Map = occGrid;
stateValidator.ValidationDistance = 0.05;
	validatorOccupancyMap oluşturularak hava aracının engellerle arasındaki mesafe belirlenmiştir.

planner = plannerRRT(ss,stateValidator); planner.MaxConnectionDistance = 5.0;	Yol planlayıcıyı oluşturulmuş ve daha fazla durumu bağlamak için maksimum bağlantı mesafesini arttırılmıştır.

planner.GoalReachedFcn = @exampleHelperCheckIfGoal;
function isReached = exampleHelperCheckIfGoal(planner, goalState, newState)
isReached = false;
threshold = 0.1;
if planner.StateSpace.distance(newState, goalState) < threshold
isReached = true;
end
end
	GoalReached fonksiyonu özelleştirilmiştir. Bu fonksiyon, uygun bir yolun belirlenen bir eşik içinde hedefe ulaşıp ulaşmadığını kontrol etmektedir. 'true', hedefe ulaşıldığında fonksiyon geri dönmekte ve planlayıcı durmaktadır.

rng default [pthObj, solnInfo] = plan(planner,start,goal);	Başlangıç ve hedef arasındaki yolu planlanmıştır. Tekrarlanabilir sonuçlar için rastgele sayı üreteci sıfırlanmıştır.
show(occGrid) hold on plot(solnInfo.TreeData(:,1),solnInfo.TreeData(:,2),'.-'); interpolate(pthObj,300) plot(pthObj.States(:,1),pthObj.States(:,2),'r-','LineWidth',2) plot(start(1),start(2),'ro') plot(goal(1),goal(2),'mo')	solnInfo verisinin içerisinde interpolasyon yapılarak hedefe bir yol çizilmiştir. Çizilen yol haritada gösterilmiştir[11].

hold off	

II. Hava aracının gitmesi gereken koordinatları hangi veri yapısından öğrendiniz?

“sehir.a.veri“ dizisi, Cezeri’nin kullanılacağı şehir için planlanmıştır. Burada kullanıcının gitmek istediği yer input olarak alınacak ve bu dizi sayesinde RRT algoritmasının hedef noktasının matris formunda konumu elde edilecektir. Örneğin, kullanıcı programın başlangıcında ‘Nereye gitmek istiyorsunuz?’ sorusuna ‘Okul’ olarak yazarsa “sehir.a.veri “ dizisinden okulun matris formunda koordinatları program içinde hedef nokta olarak atanacaktır.

III. Hava aracının inişe geçmesi için gerekli koşul seçimleriniz nelerdir?

Güvenli iniş için gereken koşullar iniş pistinin boş olması, iniş takımlarınlarının sağlam olması, iniş pistinde kanat boyu mesafesinde etki edebilecek engelin bulunmaması, iniş pistinin zemin sertliğinin uygun olması, iniş için doğru açının sağlanmasıdır (Glide Path sistemi ile iniş noktasına doğru açıyla alçalmasını ve inmesini sağlanmaktadır.). İniş için belirlenen noktaların tercih edilmesi gerekmektedir kişisel mülk ve yasak bölgelere iniş yapılmamalıdır. İniş ve kalkıştaki en büyük problemlerden biri de rüzgardır. Rüzgar herhangi bir açıdan eserek araca hem fazladan motor gücü kullanımı hem de araçta savrulma meydana getirebilir. Rüzgarın hava araçları için tehlikeli olmasını sağlayan asıl unsur yan rüzgar hızı(crosswind velocity)’dır. Her hava aracının bir yan rüzgar limiti vardır. Yan rüzgar aracın sağından veya solundan dik bir şekilde etki eden rüzgardır. Rüzgar araca tam dik olarak etki etmiyorsa yan rüzgar bileşeninin hesaplanması gerekmektedir. Yüzey rüzgarının araca paralel olan bileşeni ise önden estiği durumlarda sürtünmeye ve güç kaybına neden olsa da artan hava akış hızı sebebiyle pervanelerden üretilen kaldırma kuvvetini arttıracaktır. Bu durum sürtünmeden elde edilen güç kaybının bir telafisi olarak görülmektedir.

IV. İnişte hangi sensörü irtifa tespiti için kullandınız?

İnişte, irtifa tespiti için radar altimetresi kullanılmıştır. Çalışma prensibi, hava aracından düşey olarak gönderilen dalganın yerden yansıyıp dönme süresinin ölçülmesi ve buna bağlı olarak da irtifasının bulunmasına dayanmaktadır. Radar altimetreler uçak, helikopter, iha, füzeler gibi sivil ve askeri araçların irtifalarını bulmak için aktif olarak kullanılmaktadır.

BÖLÜM-3

Bu bölümde, şartnamede yer alan Senaryo-4’te yasak bölgeden kaçış için tasarlanan algoritma için final tasarım şablonundaki sorular sırasıyla cevaplandırılmıştır.

I. Yasak bölgeleri tespit ettiğiniz veri yapısı nedir?

"sehir.a.obstacle" dizisi, Cezeri'nin kullanılacağı şehir için uçuşa yasak bölgelere dair bilgi vermektedir. Burada yasak bölgelerin haritalama yapılırken belirlenen matrisleri bulunmaktadır. Bu dizi sayesinde RRT algoritmasının hedef noktaya gidişinde kullanacağı rastgele noktalamalar yasak bölgelerin dışında belirlenecektir.

II. Bu bölgelere girmemek için tasarladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

Algoritma	Açıklama
obstacles = csvread('sehir.a.obstacle.csv');	A şehri için hazırlanan uçuşa yasak bölgeler matris formunda .csv formatıyla programa alınmıştır.

centerX = obstacles(:,1);
centerY = obstacles(:,2);
radius = obstacles(:,3)/2;
obstacleCenterX = zeros(1, length(obstacles));
obstacleCenterY = zeros(1, length(obstacles));
obstacleRadius = zeros(1, length(obstacles));
for i = 1 : length(obstacles)
obstacleCenterX(i) = centerX(i);
obstacleCenterY(i) = centerY(i);
obstacleRadius(i) = radius(i);
end
	Uçuşa yasak bölgelerin matris formuna çevrilip tanımlanması yapılmıştır.

for i = 1 : length(obstacles)
a=linecirc(slope,yIntercept,obstacleCenterX(i),obstacleCenterY(i),obstacleRadius(i));
if (~isnan(a))
addCondition = 0;
break
else
addCondition = 1;
end
end
if xNew(1)>= 0.5
	Hedefe ulaşılırken kullanılan rastgele noktalamaların birleşimiyle oluşan çizginin yasak bölgelere gelmesi durumunda yeni bir nokta değeri atanarak RRT algoritiması için farklı bir yol oluşturulmaktadır[12].

xNew(1) = 0.5;
end
if xNew(2)>= 0.5
xNew(2) = 0.5;
end	

BÖLÜM-4

Bu bölümde, şartnamede yer alan Senaryo-7'de trafikte olan diğer uçan arabaları gözeterek kaza yapmadan uçuşu sağlayan algoritma için final tasarım şablonundaki sorular sırasıyla cevaplandırılmıştır.

I. Diğer hava araçlarının konumlarını hangi veri yapısından öğrendiniz?

Diğer hava araçlarının konumları, 'objectDetection' veri yapısından alınmaktadır. Bu veri yapısıyla hava aracının hareket esnasında sensörlerden ve kameradan alınan veriler saniyede 20 kere hız ve dönüş hızı hesaplaması yapılarak kazalar önlenmeye çalışılmıştır.

II. Hava araçları ile kaza yapmamak için nasıl bir algoritma tasarladınız? Algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

Algoritma	Açıklama
[videoReader, videoDisplayHandle, bepPlotters, sensor] = helperCreateFCWDemoDisplay('a_city_c2s_fcw_10s.mp4', 'SensorConfigurationData.mat');	Hava aracının etrafındaki kameralar ve sensör verileri alınmıştır.
[visionObjects, radarObjects, inertialMeasurementUnit, laneReports,timeStep, numSteps] =readSensorRecordingsFile('cezeri_collision_sensor.mat');	Kamerada kaydedilen nesne algılamaların dosyası okunmuştur.
laneWidth = 3.75; egoLane = struct('left', [0 0 laneWidth/2], 'right', [0 0 -laneWidth/2]);	Bir başlangıç ego şeridi hesaplanmıştır. Kaydedilen şerit bilgisi geçersizse, şerit sınırlarını, aracın her iki tarafında yarım şerit mesafesi olan düz çizgiler olarak tanımlanmıştır.
time = 0; currentStep = 0;	Zaman değişkenleri tanımlanmıştır.

snapTime = 9.3;	

[tracker, positionSelector, velocitySelector] = setupTracker();
while currentStep < numSteps && ishghandle(videoDisplayHandle)
currentStep = currentStep + 1;
time = time + timeStep;
	İzleyici başlatılıp senaryo sayaçları güncellenmiştir.

[detections, laneBoundaries, egoLane] = processDetections(... visionObjects(currentStep), radarObjects(currentStep), ... inertialMeasurementUnit(currentStep), laneReports(currentStep), ... egoLane, time);	Sensör algılamaları kullanıcıya objectDetection girdileri olarak işlenmiştir.
confirmedTracks = updateTracks(tracker, detections, time);	ObjectDetections listesi ile zamana güncellenip parçalar geri getirilmiştir.
mostImportantObject = findMostImportantObject(confirmedTracks, egoLane, positionSelector, velocitySelector);	En önemli nesne tespit edilip ve önden çarpışma uyarısı hesaplanmıştır.

frame = readFrame(videoReader);
helperUpdateFCWDemoDisplay(frame, videoDisplayHandle, bepPlotters, ...
laneBoundaries, sensor, confirmedTracks, mostImportantObject, positionSelector, ...
velocitySelector, visionObjects(currentStep), radarObjects(currentStep));
	Video ve kuş bakışı çizim ekranları güncellenmiştir.

if time >= snapTime && time < snapTime + timeStep snapnow; end end	Anlık görüntü yakalanmıştır[13].

BÖLÜM-5

Bu bölümde, şartnamede yer alan Senaryo-8'de en yakın hastaneyi bulmayı sağlayan algoritma için final tasarım şablonundaki sorular sırasıyla cevaplandırılmıştır.

I. Hastanelerin olduğu veri yapısı nedir?

“sehir.a.hospitals.veri“ dizisi, Cezeri’nin kullanılacağı şehir için planlanmış, haritalamayı matris formuna dönüştürmek için kullanılan veri bütünüdür. Şehir içerisindeki hastane ve okul gibi her bir konumun haritalama düzeninde matris karşılığını barındırmaktadır.

II. En yakın hastanenin seçimini yaptığınız algoritmayı EK-1’de verilen örnekteki gibi açıklayınız.

Algoritma	Açıklama
y_hosp= csvread('sehir.a.hospitals.csv');	Şehirdeki hastanelerin matris koordinatları çekilmiştir.
x_hosp= startNode;	Başlangıç noktası belirlenmiştir.
[mIdx,mD] = knnsearch(X,Y,'K',10,'Distance','minkowski','P',5); [cIdx,cD] = knnsearch(X,Y,'K',10,'Distance','chebychev');	Minkowski ve Chebychev mesafe ölçümleri kullanılarak X ile Y noktaları arasında bir knnsearch gerçekleştirilmiştir.
gscatter(X(:,1),X(:,2),species) line(Y(:,1),Y(:,2),'Marker','x','Color','k',... 'Markersize',10,'Linewidth',2,'Linestyle','none') line(X(mIdx,1),X(mIdx,2),'Color',[.5 .5 .5],'Marker','o',... 'Linestyle','none','Markersize',10) line(X(cIdx,1),X(cIdx,2),'Color',[.5 .5 .5],'Marker','p',... 'Linestyle','none','Markersize',10) legend('setosa','versicolor','virginica','query point',... 'minkowski','chebychev','Location','best')	En yakın komşu aramasının sonuçları görselleştirilmiştir. Minkowski'nin en yakın komşularını belirtmek için daireler kullanılmıştır. Chebychev'in en yakın komşularını belirtmek için pentagramları kullanılmıştır[14].

BÖLÜM-6

Bu bölümde, şartnamede yer alan Senaryo-10 içerisinde GNSS arızası yaşandığında inişi tamamlayan algoritma için final tasarım şablonundaki sorular sırasıyla cevaplandırılmıştır.

I. GNSS arızasını hangi veri yapısından tespit ettiniz?

‘posd.csv’ veri yapısı GNSS arızalarının bulunduğu veri yapısıdır. GNSS sensöründe aykırı değerleri kaldırma filtresinin deaktif olduğu veri çıktıları bu dizinde kaydedilmektedir.

II. Bu arıza sonrası hava aracının irtifasını hesapladığınız algoritmayı EK-1’de verilen örnekteki gibi açıklayınız.

Algoritma	Açıklama

p = inputParser;
fName = 'densityalt';
p.FunctionName = fName;
p.addRequired('rho',@(x) validateattributes(x,{'numeric','DimVar'},...
{'positive'},'','density, rho,'));
	Giriş verileri ayrıklaştırılmıştır.

p.addParameter('inputUnits','SI');
p.addParameter('outputUnits','SI');
p.addParameter('atmosphereFunction','atmos');
p.addParameter('atmosphereArgs',{},@iscell);
p.addParameter('method','pchip');
p.addParameter('hMin',0,@(x) validateattributes(x,{'numeric'},...
{'scalar','finite','real'},'','altitude minimum h (m)'));
p.addParameter('hMax',86000,@(x) validateattributes(x,{'numeric'},...
{'scalar','finite','real'},'','altitude maximum h (m)'));
	Olasılıklar, 'fzero' veya 'biseksiyon' veya bir interp1 yönteminin tam eşleşmeleridir. interp1 tarafından doğrulama.

p.addParameter('options',{})	Yalnızca fzero/iki bölüm arama yöntemiyle kullanılmaktadır.

p.addParameter('spacing',50,@(x) validateattributes(x,{'numeric'},...
{'positive','scalar','finite','real'},'','altitude grid spacing (m)'));
parse(p,rho,varargin{:});
i = p.Results;
rho = i.rho;
if ~iscell(i.options)
i.options = {i.options};
end
	Yalnızca interpolasyon yöntemiyle kullanılmaktadır.

i.inputUnits = validatestring(i.inputUnits,...	Giriş dizelerini doğrulanmıştır.

'SI','kg/m3','kg/m^3','US','slug/ft3','slug/ft^3'},fName,'inputUnits'); i.outputUnits = validatestring(i.outputUnits,... {'SI','m','meters','US','ft','feet'},fName,'outputUnits'); i.atmosphereFuntion = validatestring(i.atmosphereFunction,... {'atmos','stdatmo','tropos','atmosisa','atmoscoesa','atmosnonstd'},... fName,'atmosphereFunction');	

dimVarOut = false;
if isa(rho,'DimVar')
rho = rho/(u.kg/u.m^3);
dimVarOut = true;
elseif any(strcmpi(i.inputUnits,{'US','slug/ft3','slug/ft^3'}))
rho = rho * 515.3788183931961;
end
	Birim dönüştürme işlemiyle SI birim sistemine çevrim yapılmıştır.

function rho = myAtmo(h) switch i.atmosphereFunction case {'stdatmo' 'atmos' 'tropos'} rho = feval(i.atmosphereFunction,h,i.atmosphereArgs{:}); otherwise [~,~,~,rho] = feval(i.atmosphereFunction,h,i.atmosphereArgs{:}); end end	Atmosfer modeli oluşturularak girdi olarak yalnızca yüksekliği (metre olarak) alan ve ilk ve tek çıktı olarak yoğunluğu (kg/m^3 olarak) döndüren bir atmosfer işlevi oluşturulmuştur[15].
switch i.method case 'fzero' h = fzero(@(h) rhomyAtmo(h),[i.hMin,i.hMax],i.options{:}); case 'bisection'	Arama ve interpolasyon yöntemleri arasında geçiş yapılmıştır.

h = bisection(@myAtmo,i.hMin,i.hMax,rho,i.options{:});
otherwise
H = (i.hMin:i.spacing:i.hMax)';
RHO = myAtmo(H);
h = interp1(RHO,H,rho,i.method);
end	

if dimVarOut
h = h*u.m;
elseif any(strcmpi(i.outputUnits,{'US','ft','feet'}))
h = h / 0.3048;
end
end
	İrtifa metre olarak hesaplanmıştır.

BÖLÜM-7

Bu bölümde, şartnamede yer alan Senaryo-11 içerisinde GNSS arızası yaşandığında seyir yapmaya devam eden ve görevi tamamlamayı sağlayan algoritma için final tasarım şablonundaki sorular cevaplandırılmıştır.

I. Bu arıza sonrası hava aracının pozisyonunu(yatay) hesapladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

Algoritma	Açıklama
vehicleDims=csvread('vehicle.dimensions.csv')	Hava aracının ölçüleri program içerisine alınmıştır.
approxSeparation = 0.5; numSmoothPoses = round(refPath.Length / approxSeparation);	Yaklaşık 0,5 m'lik bir ayrım kullanarak döndürülecek poz sayısını belirlenmiştir.
[refPoses, directions, cumLengths, curvatures] = smoothPathSpline(transitionPoses, directions, numSmoothPoses);	Düzgün yol boyunca ayrık pozlar dönüştürülmüştür.
vehicleSim = HelperVehicleSimulator(costmap, vehicleDims);	Araç simülatörü oluşturulmuştur.

vehicleSim.setVehiclePose(currentPose);
currentVel = 0;
vehicleSim.setVehicleVelocity(currentVel);
	Araç pozu ve hızı ayarlanmıştır.

reachGoal = false;
while ~reachGoal
[refPose, refVel, direction] = pathAnalyzer(currentPose, currentVel);
updateDrivingDirection(vehicleSim, direction);
	Yoldaki referans pozu ve buna karşılık gelen hızı hesaplanmıştır. Simülatör için sürüş yönünü güncellenmiştir.

steeringAngle = lateralControllerStanley(refPose, currentPose, currentVel, ... 'Direction', direction, 'Wheelbase', vehicleDims.Wheelbase);	Hesaplama yönlendirme komutu çalıştırılmıştır. Yanmal pozisyonm için Stanley fonksiyonu kullanılmıştır.
reachGoal = helperGoalChecker(goalPose, currentPose, currentVel, endSpeed, direction); waitfor(controlRate);	Aracın hedefe ulaşıp ulaşmadığını kontrol edilip sabit oranlı yürütmeyi beklenmiştir.
currentPose = getVehiclePose(vehicleSim);	Aracın yanal pozisyonu alınmıştır[16].

BÖLÜM-8

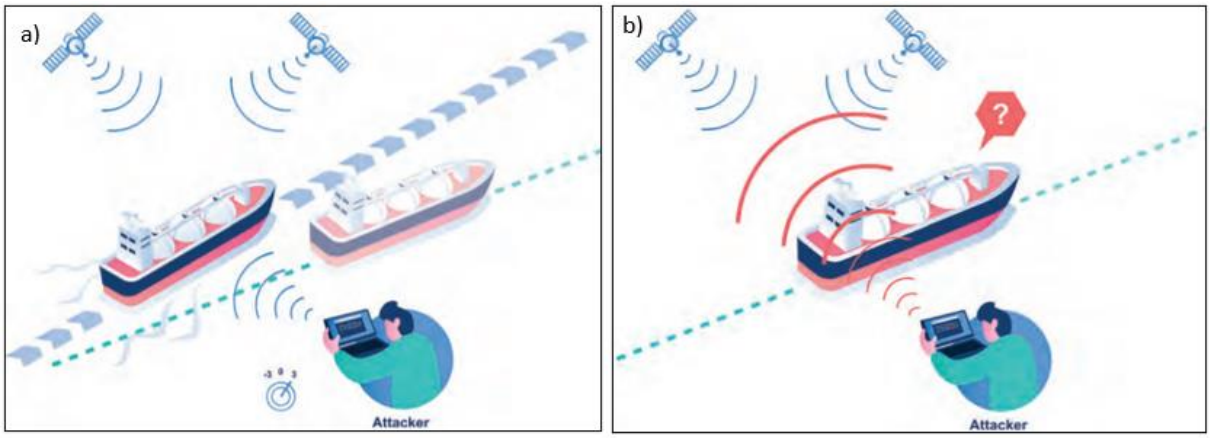
Bu bölümde, şartnamede yer alan Senaryo-12'de GNSS arıza durumunu tespit eden algoritma için final tasarım şablonundaki sorular sırasıyla cevaplandırılmıştır.

I. GNSS karıştırma (jamming) ve yanıltma (spoofing) nedir? Farkları nelerdir?

GNSS sinyallerinin gücü düşüktür, bu da zayıf bir girişim kaynağının alıcının arızalanmasına veya tehlikeli şekilde yanıltıcı bilgi üretmesine neden olabileceği anlamına gelmektedir. GNSS Karıştırma(jamming), radyo iletişimini engellemek veya müdahale etmek için bir frekans verici cihaz kullanma işlemidir. Bu işlem uzay hava durumu, L1 frekansında sinyal yayan ve GNSS sinyal alımını karıştırabilen hatalı ekipman da dahil olmak üzere, kasıtsız araçlardan da kaynaklanabilmektedir. Kasıtlı karıştırmada, askeri bozucuların yanı sıra Kişisel Koruma Cihazları (PPD) gibi stratejiler de sıklıkla kullanılmaktadır.

GNSS Yanıltma(spoofing), yerel olarak iletilen ve alıcıyı olmadığı yerde olduğunu düşünmesine sebep olarak yanıltmak için kodlanan GNSS benzeri sinyallerin sağlanmasıdır. GNSS Yanıltma(spoofing) saldırısı, bir dizi normal GNSS sinyaline benzeyecek şekilde

yapılandırılmış yanlış GNSS sinyalleri yayınlayarak veya başka bir yerde veya farklı bir zamanda yakalanan orijinal sinyalleri yeniden yayınlayarak yapılmaktadır. GNSS yanıltma saldırısının yaygın bir biçimi "taşıma saldırısı (carry-off attack)" olarak adlandırımaktadır. Taşıma saldırısında sinyaller hedef alıcı tarafından gözlemlenen gerçek sinyallerle senkronize olarak yayınlanarak başlamakta ve ardından sahte sinyallerin gücünün kademeli olarak artırılmasıyla yapılmaktadır[17].



Şekil 1 a) GNSS Yanıltma (Spoofing) b) GNSS Karıştırma(Jamming) Şematik Gösterimi

GNSS karıştırma kasıtlı veya kasıtsız olarak yapılabilmektedir ve kasıtlı karıştırmada kullanılacak ekipmanlar basit, kolay elde edilebilir ve ucuzdur. GNSS yanıltma, özellikle saldırının gizli kalması gerekiyorsa GNSS karıştırmadan çok daha komplekstir. GNSS yanıltma ekipmanları nispeten daha pahalı ve sadece askeri alanda kullanımına izin verilmektedir. GNSS karıştırma saldırısının GNSS yanıltma saldırısına göre tespit edilmesi daha kolaydır.

II. GNSS yanıltma (spoofing) durumunu tespit ettiğiniz algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

Algoritma	Açıklama
import pandas as pd import math from sklearn.metrics import mean_squared_error import matplotlib.pyplot as plt import tensorflow as tf from sklearn.preprocessing import MinMaxScaler gpus = tf.config.experimental.list_physical_devices('GPU') for gpu in gpus:	GNSS veri seti oluşturulmuştur. CAN ve IMU verileri alınmıştır. Zaman işlenmiştir. Mesafe hesabı yapılmıştır. Veri kümeleri birleştirilerek GNSS hızı ile CAN hızını

tf.config.experimental.set_memory_growth(gpu, True)
def average(seq, total=0.0):
num = 0
for item in seq:
total += item
num += 1
return total / num
if __name__ == '__main__':
CSV_FILE_PATH=
'D:\\comma2k19\\Chunk_03\\99c94dc769b5d96e_2018-05-01--08-13-53.csv'
df = pd.read_csv(CSV_FILE_PATH)
values = df.to_numpy()
times = values[:, -1]
distance = values[:, -2]
model = tf.keras.models.load_model('lstm.model')
test_X = values[:, :3]
scaler = MinMaxScaler(feature_range=(0, 1))
test_X = scaler.fit_transform(test_X)
test_X = test_X.reshape((test_X.shape[0], 1, test_X.shape[1]))
test_y = distance
yhat = model.predict(test_X)[:, 0]
rmse = math.sqrt(mean_squared_error(yhat, test_y))
print('Test RMSE: %.3f' % rmse)
scores = model.evaluate(test_X, test_y)
rmse = math.sqrt(mean_squared_error(yhat, test_y))
plt.plot(times, yhat, label='prediction')
plt.plot(times, distance, label="ground_truth")
plt.title('Comparison between truth and prediction', fontsize=18)
plt.xlabel('Boot time (s)', fontsize=18)
plt.ylabel('Distance travelled during single timestamp (m) ', fontsize=12)
	karşılaştırılıp sapma kontrolü yapılmıştır. Sapma olması durumunda sapma miktarı grafik çıktısıyla görüntülenmiştir[18].

```
plt.legend()
plt.show()
min = min((distance - yhat), key=abs)
max = max((distance - yhat), key=abs)
avr = average(distance-yhat)
print('Min:%f' % min)
print('Max:%f' % max)
print('average:%f' % avr)
```

BÖLÜM-9

Bu bölümde, şartnamede yer alan Senaryo-13'te inişi tamamlamak için kullanılacak sensör seçimi yapılan algoritma için final tasarım şablonundaki sorular sırasıyla cevaplandırılmıştır.

I. İniş için hangi sensörü irtifa kaynağı olarak seçtiniz? Neden?

İnişte, irtifa kaynağı olarak radar altimetresi seçilmiştir. Radar altimetreleri, askeri ve sivil hava araçlarında özellikle düşük görüş koşullarında, otomatik inişlerde yaklaşma ve iniş için sıklıkla kullanılmaktadır. Aynı zamanda otomatik pilotun parlama manevrasına ne zaman başlayacağını bilmesini sağlamaktadır. Radar altimetreleri, uçuş bilgisayarının bir parçası olan otomatik gaz kelebeğine veri vermektedirler. Çalışma prensipleri hava aracından düşey olarak gönderilen dalganın yerden yansıyıp dönme süresinin ölçülmesi ve buna bağlı olarak da irtifasının bulunmasına dayanmaktadır. Radar altimetresinin irtifa ölçme menzilinin kısa olması sebebiyle iniş için daha verimli olmaktadırlar.

II. Diğer sensörleri neden seçmediniz?

Barometerik altimetreler atmosferik basıncın yükseklikle doğru orantılı olarak azalması ilkesine dayanarak çalışmaktadır. Yükseklik ölçümü için dış basınç ile iç basınç karşılaştırılmaktadır. Çünkü hava aracının irtifasına göre, dış ortamdan alınan havanın basıncı değişecektir. Dış ortamdan hava almak için, hava aracının çeşitli bölgelerinde "statik port" adı verilen boşluklar bulunmaktadır. Barometrik altimetreler düz uçuşlar için daha verimlidir bu nedenle tercih edilmemiştir.

Optik altimetreler, LiDAR gibi enerji temelli teknolojilerle benzerlik göstermektedir. Bir verici aracılığıyla lazer ışını formunda enerji dalgaları göndermektedirler. Enerji dalgalarının yer de dahil olmak üzere çarptığı en yakın objeden geri dönmesiyle hava taşıtının irtifasını, etrafındaki ve altındaki yüzeyleri (dağlar, binalar vb.) belirlemektedirler. Yüksek kesinliğin sağlanması için genellikle, GPS ve atalestel navigasyon sistemleri (INS) ile birlikte kullanılmaktadır[19].

Kabin altimetresi, basınçlandırılmış tüm hava araçlarında bulunmaktadır ve kabin basıncını feet olarak yükseklik cinsinden göstermektedir. Pilot bu göstergeye yardımıyla kabin basıncının limitler içinde olup olmadığını kontrol etmektedir. Bu nedenle iniş için kullanılması uygun görülmemiştir[20].

III. Bu seçimi yapan algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

Algoritma	Açıklama

Fs = 100;
duration = 60*10;
numSamples = duration*Fs;
	10 dakikalık bir simülasyon için sabit tanımlamaları yapılmıştır.

altimeter = altimeterSensor('SampleRate',Fs, 'ConstantBias',0.01, ... 'NoiseDensity',0.05, ... 'BiasInstability',0.05, ... 'DecayFactor',0.5);	Hava aracının sabit ön frekansı 0.01, gürültü yoğunluğu 0.05, ön süreksizlik faktörü 0.05 ve çürüme faktörü 0.5 alınarak altimetre sensörünün okuma verileri düzenlenmiştir[21].
trueAltitude = zeros(numSamples,3);	Altimetre sensörü için çıkış verisi tanımlamıştır.
if altitude<760;	Radar altimetresinin max irtifa ölçüm menzilinin 760 m olduğu gözetilerek şart tanımlanmıştır.
altimeterSensor('Referance Frame',RF)=altimeter;	760 m'nin altındaki irtifa değerinde altimetre sensörü radar altimetre sensörüne atanmaktadır.
trueAltitude_new=trueAltitude;	İniş için yeni bir irtifa değeri hesaplanmaktadır.
altimeterReadings = altimeter(trueAltitude_new);	Radar altimetreden okunan irtifa değerleri yeniden yazılmaktadır.

BÖLÜM-10

Bu bölümde, şartnamede yer alan Senaryo-16'da şarj durumunu gözeterek şarj istasyonlarını tespit eden ve hava aracını şarj istasyonuna yönlendiren algoritma için final tasarım şablonundaki sorular sırasıyla cevaplandırılmıştır.

I. Batarya durumlarını hangi veri yapısından öğrendiniz?

PID ile tüm bataryalar için kontrol algoritması oluşturulmuştur. Burada kullanılan 'Battery Read' fonksiyonu ile bataryaların şarj durumları okunmaktadır. 'Battery Indıcator' fonksiyonu ile kullanıcının görebileceği arayüzde batarya durumu gösterilmektedir. 'Battery Read' fonksiyonu çıktısı, 'Battery Indıcator' fonksiyonunun yanısıra bir şart komut satırıyla belirli bir gerilim eşiğinin altına inmesiyle RRT algoritmasında otomatik en yakın şarj istasyonuna yönelinecektir. 'Battery Read' fonksiyonunun çıktısı yani batarya durumu, 'battery.cond.csv' veri yapısına yazılmaktadır.

II. Batarya durumunun hangi gerilim eşiğinde şarj istasyonuna inilmesi gerektiğini seçtiniz? Neden?

Cezeri'de 8 adet BLDC motor bulunmaktadır ve motorlar gücü lityum-iyon batarya paketinden almaktadır[22]. Lityum-bataryaların ömrü, yüksek sıcaklıklarda çok daha hızlı etkisini tükenmektedir. Önerilen batarya sistemleri depolama sıcaklığı, oda sıcaklığı veya daha soğuk sıcaklıklardır. Lityum-iyon bataryalar hücre başına 2,4 V ila 3,0 V olan minimum voltajlarının altına inmemelidir[23]. Buradan gerilim eşiği değeri 21.6 V olarak seçilmiştir. Kullanıcıya batarya durumu yüzdelik olarak aşağıdaki denklem kullanılarak verilmektedir. Denklemde $V_{altseviye}$ ve $V_{üstseviye}$ bataryanın en verimli çalıştığı alt ve üst seviyelerdir.

$$LiPo_{\%} = \left(\frac{V_{lipo} - V_{altseviye}}{V_{üstseviye} - V_{altseviye}}\right) x100$$

III. Şarj istasyonlarının olduğu veri yapısı nedir?

"sehir.a.chargestat.veri" dizisi, Cezeri'nin kullanılacağı şehir için şarj istasyonlarına göre planlanmış , haritalamayı matris formuna dönüştürmek için kullanılan veri bütünüdür. Hava aracı, şarj durumu belli bir değerin altına indiğinde veya kullanıcı şarj istasyonuna gitmek istediğinde bu veri dizininden tüm şarj istasyonlarının haritalamada matris formunda konumları programa alınacaktır.

Kaynakça

[1] "Python Nedir? En Popüler Programlama Dili - Teknoloji.org." https://teknoloji.org/python-nedir/ (accessed May 03, 2022).

[2] "Python Programlama Dili Nedir? Nerelerde Kullanılır? – BlueMark Academy." https://bluemarkacademy.com/python-programlama-dili-nedir-nerelerde-kullanilir/ (accessed May 03, 2022).

[3] "MATLAB Programlama Dili Nedir? -." https://programlamadilleri.net/matlab-programlama-dili-nedir/ (accessed May 03, 2022).

[4] "MATLAB SİMULİNK İLE GÖRÜNTÜ İŞLEME -1 - Veri Bilimi Okulu - Veri

Bilimi Okulu.” https://www.veribilimiokulu.com/matlab-simulink-ile-goruntu-isleme-1/ (accessed May 03, 2022).

[5] “How Barometers Measure Air Pressure.” https://www.thoughtco.com/how-barometers-measure-air-pressure-3444416 (accessed May 03, 2022).

[6] “Aneroid Barometer- How does it Work? | Mechanism, Uses, Concerns.” https://www.toppr.com/guides/physics/mechanical-properties-of-fluids/aneroid-barometer-how-does-it-work/ (accessed May 03, 2022).

[7] “GNSS Nedir? GNSS Sistemleri Nelerdir? | Haritacıyız.” https://www.haritaciyiz.com/gnnssistemleri/ (accessed May 03, 2022).

[8] “Geoid vs Ellipsoid: What’s the Difference and Why Does it Matter?” https://enterprise-insights.dji.com/blog/geoid-vs-ellipsoid (accessed May 03, 2022).

[9] Y. Günhan, Y. Doç, and M. Demġrer, “ATALETSEL ÖLÇÜM BĠRĠMĠ HATALARININ BELĠRLENMESĠ VE DÜZELTĠLMESĠ DETECTION AND COMPENSATION OF INERTIAL MEASUREMENT UNIT ERROR.”

[10] “GPS Navigation Toolbox - File Exchange - MATLAB Central.” https://www.mathworks.com/matlabcentral/fileexchange/41364-gps-navigation-toolbox (accessed May 03, 2022).

[11] “RRT Kullanarak Mobil Robot Yollarını Planlayın - MATLAB & Simulink.” https://www.mathworks.com/help/nav/ug/plan-mobile-robot-paths-using-rrt.html (accessed May 03, 2022).

[12] “GitHub - kucar17/RRT-Path-Planning-Algorithm: This script is designed for the demonstration of a Sampling Based RRT Path Planning which creates a path for a robot in an environment which consists of obstacles.” https://github.com/kucar17/RRT-Path-Planning-Algorithm (accessed May 03, 2022).

[13] “Forward Collision Warning Using Sensor Fusion - MATLAB & Simulink.” https://www.mathworks.com/help/driving/ug/forward-collision-warning-using-sensor-fusion.html (accessed May 04, 2022).

[14] “Find k-nearest neighbors using input data - MATLAB knnsearch - MathWorks United Kingdom.” https://uk.mathworks.com/help/stats/knnsearch.html (accessed May 04, 2022).

[15] “Density altitude calculator - File Exchange - MATLAB Central.” https://www.mathworks.com/matlabcentral/fileexchange/39325-density-altitude-calculator (accessed May 05, 2022).

[16] “Automated Parking Valet - MATLAB & Simulink.”

https://www.mathworks.com/help/driving/ug/automated-parking-valet.html (accessed May 05, 2022).

[17] “Jamming and Spoofing of Global Navigation Satellite Systems (GNSS) 9 Jamming and Spoofing of Global Navigation Satellite Systems (GNSS),” 2019.

[18] “GitHub - Juntu-hub/Prediction-Based-GNSS-Spoofing-Attack-Detection-for-Autonomous-Vehicle: experimental realization for Prediction-Based GNSS Spoofing Attack Detection for Autonomous Vehicle in python.” https://github.com/Juntu-hub/Prediction-Based-GNSS-Spoofing-Attack-Detection-for-Autonomous-Vehicle#readme (accessed May 04, 2022).

[19] “Altimetre Nedir? Altimetre Nasıl Çalışır? - Teknoloji.org.” https://teknoloji.org/altimetre-nedir-altimetre-nasil-calisir/ (accessed May 04, 2022).

[20] “Altimetre Nedir? Nasıl Çalışır? Ne İşe Yarar? | Teknoloji Projeleri.” https://teknolojiprojeleri.com/teknik/altimetre-nedir-nasil-calisir-ne-ise-yarar (accessed May 04, 2022).

[21] “Altimeter simulation model - MATLAB.” https://www.mathworks.com/help/nav/ref/altimetersensor-system-object.html (accessed May 05, 2022).

[22] “BAYKAR İnsansız Hava Aracı Sistemleri | Uçan Araba.” https://baykartech.com/tr/ucan-araba/ (accessed Dec. 01, 2021).

[23] “Lithium-Ion Information Guide | Houston, Texas USA |.” https://www.swe.com/lithiumion/ (accessed May 05, 2022).

TEKNOFEST
İSTANBUL HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ